



ASSIGNMENT ON STRUCTURED PROGRAMMING LAB

Emergency Potable Water Allocation for a Flood-Affected Municipality

Student Name: 1. AL ARAF BIN ALAM
2. MARUF ABID JOTI

Student ID: 1. 26103163
2. 26103239

Programme: Bachelor of Computer Science and Engineering (BCSE)

Course Code: CSC 1384

Course Title: Structured Programming LAB

Section: D

Course Instructor: Abdullah Mohammad Sakib

Submission Date: 6 September 2026

2. Problem Analysis

The assignment addresses an emergency potable-water allocation problem created by severe flooding. A temporary municipal distribution centre has 13,500 litres of potable water available for six affected service zones, while the total requested quantity is greater than the available supply. Therefore, the programme must decide how limited water should be distributed without relying on a fixed order of request. The official task specifically requires the student to identify relevant decision factors and formulate an allocation strategy that balances emergency need, fairness and efficient utilisation of water.

The six service zones differ in service type, requested quantity, minimum requirement, distribution loss and previous waiting cycles. These attributes create competing priorities. Hospitals and emergency services require dependable minimum supply; shelters serve displaced people; residential areas may have accumulated waiting time; and higher distribution losses reduce the amount that finally reaches the destination. A useful computational solution must therefore consider more than one variable at the same time.

2.1 Original Data Set

Zone ID	Service Zone	Type	Requested (L)	Minimum (L)	Loss	Waiting
Z01	Municipal Hospital	Hospital	4000	3000	2%	0
Z02	Central Flood Shelter	Emergency Shelter	3500	2500	4%	1
Z03	Ward 3	Residential	4500	2000	8%	2
Z04	Ward 5	Residential	3800	1800	12%	3
Z05	School Emergency Shelter	Emergency Shelter	2600	1600	5%	1
Z06	Fire and Emergency Service	Emergency Service	2000	1500	1%	0

The total requested amount is 20,400 L. Because only 13,500 L is available in the original case, the programme cannot satisfy all full requests simultaneously. The problem is therefore one of constrained allocation: the programme must decide which zones should be served first and how much can be released while respecting the available quantity.

2.2 Major Decision Factors

- Requested quantity indicates the total amount a zone would ideally receive. Larger requests represent larger unresolved demand when supply is constrained.
- Minimum requirement represents a lower service threshold. The allocation logic attempts to satisfy this threshold before giving water to lower-priority zones, subject to availability.
- Service-zone type captures urgency. The implementation assigns higher base urgency to hospitals and emergency services, a slightly lower value to emergency shelters, and a lower value to residential areas.
- Previous waiting cycles provide a fairness component. A zone that has remained underserved for more cycles receives additional priority.
- Distribution loss represents water that is lost between release and destination. A high loss percentage makes a release less efficient and therefore creates a small penalty in the priority score.
- Unresolved demand is represented in the priority formula through the gap between requested water and minimum requirement. A larger gap increases the demand component.
- Shortage is the remaining difference between the requested quantity and the effective water actually delivered after losses. It is used after allocation to classify final service condition and identify the highest unresolved requirement.

2.3 Conflicting Requirements

A first-come-first-served approach would ignore differences in urgency and accumulated waiting. Equal distribution would treat a hospital, an emergency shelter and a residential area as if they had the same emergency role. A fixed priority list would be rigid and would not respond appropriately when the input values

change. The assignment instead asks for a strategy that remains functional when values are changed, so the decision logic must be calculated from the supplied data rather than hard-coded for one dataset.

- Emergency requirement versus fairness: critical services may need immediate water, but repeatedly starving residential zones is unfair.
- Complete supply versus wider coverage: fully satisfying a small number of zones can leave other affected populations unserved, while spreading water too thinly can leave critical facilities below minimum.
- Distribution efficiency versus social requirement: a high-loss zone is less efficient to supply, but excluding it solely because of loss would ignore the people depending on that zone.
- Previous waiting versus new emergency: a long waiting period can justify increased priority, but a newly urgent service may still need a high base priority.
- Accuracy versus computational efficiency: a decision model should represent the relevant factors without introducing unnecessary repeated calculations or redundant data handling.

2.4 Problem Constraints

- Total water allocated must never exceed the available water.
- All six zones must be processed systematically.
- The method must continue working when input values are changed.
- Ties in priority must be resolved consistently.
- Distribution loss must be reflected in the amount released and in the effective delivered amount.
- The final report must explain the strategy, implementation and validation evidence.

3. Proposed Solution

The programme uses a weighted priority score followed by a descending sort. This strategy is a design choice made because the assignment intentionally does not prescribe a fixed priority formula. The actual C source defines the score using urgency, waiting, the requested-minus-minimum demand gap, and a small loss penalty.

3.1 Priority Formula

Priority = Urgency + Waiting Score + Demand Score - Loss Penalty

Waiting Score = Waiting Cycles x 2.0

Demand Score = (Requested Water - Minimum Requirement) / 500.0

Loss Penalty = Distribution Loss (%) x 0.10

Therefore:

Priority = Urgency + (Waiting Cycles x 2.0)
+ [(Requested - Minimum) / 500]
- (Loss x 0.10)

The constants 2.0, 500.0 and 0.10 are not provided by the assignment specification; they are the weighting decisions encoded in the submitted programme. The design gives service type the strongest direct influence, adds a meaningful but controlled reward for waiting, increases priority for zones with a larger gap above their minimum requirement, and slightly reduces priority when distribution loss is high.

3.2 Urgency Values

Service Type	Urgency Score	Rationale in the Programme
Hospital	10	Highest emergency importance
Emergency Service	10	Highest emergency importance
Emergency Shelter	8	High emergency/social need
Residential	5	Lower base urgency but affected by waiting/demand

3.3 Worked Priority Calculations

Z04 - Ward 5

Urgency = 5; Waiting = 3; Loss = 12%; Requested = 3800; Minimum = 1800.

$$\begin{aligned}\text{Priority} &= 5 + (3 \times 2.0) + ((3800 - 1800)/500) - (12 \times 0.10) \\ &= 5 + 6 + 4 - 1.2 \\ &= 13.80.\end{aligned}$$

Z03 - Ward 3

Urgency = 5; Waiting = 2; Loss = 8%; Requested = 4500; Minimum = 2000.

$$\begin{aligned}\text{Priority} &= 5 + (2 \times 2.0) + ((4500 - 2000)/500) - (8 \times 0.10) \\ &= 5 + 4 + 5 - 0.8 \\ &= 13.20.\end{aligned}$$

Z01 - Municipal Hospital

Urgency = 10; Waiting = 0; Loss = 2%; Requested = 4000; Minimum = 3000.

$$\begin{aligned}\text{Priority} &= 10 + 0 + ((4000 - 3000)/500) - (2 \times 0.10) \\ &= 10 + 2 - 0.2 \\ &= 11.80.\end{aligned}$$

3.4 Priority Order for the Original Case

Order	Zone	Priority
1	Z04 - Ward 5	13.80
2	Z03 - Ward 3	13.20
3	Z01 - Municipal Hospital	11.80
4	Z02 - Central Flood Shelter	11.60
5	Z05 - School Emergency Shelter	11.50
6	Z06 - Fire and Emergency Service	10.90

3.5 Distribution-Loss Formulas

Full release requirement = Requested Water / (1 - Loss% / 100)

Minimum release requirement = Minimum Water / (1 - Loss% / 100)

Division by the retained fraction is necessary because the amount released at the distribution centre is larger than the quantity finally received. For example, with 12% loss, only 88% of the released amount reaches the destination. Therefore, to deliver exactly 3,800 L, the release must be $3,800 / 0.88 = 4,318.18$ L rather than $3,800 \times 0.88$.

Loss Amount = Allocated Water $\times$ Loss% / 100

Effective Water Delivered = Allocated Water - Loss Amount

Remaining Shortage = Requested Water - Effective Water Delivered

The programme then prevents a negative shortage by setting it to zero when the computed value is below zero. Service status is FULL when effective delivery reaches the request, MINIMUM when it reaches the minimum requirement, BELOW MIN when some water is delivered but the minimum is not reached, and UNSERVED when no water is delivered.

3.6 Worked Distribution Example

For Z04, the original requested quantity is 3,800 L and the distribution loss is 12%. The release needed for full delivery is $3,800 / (1 - 0.12) = 3,800 / 0.88 = 4,318.18$ L. The resulting loss is $4,318.18 \times 0.12 = 518.18$ L,

leaving approximately 3,800.00 L delivered. The same principle is applied to the minimum requirement: $1,800 / 0.88 = 2,045.45$ L.

3.7 Allocation Procedure

1. Reset previously calculated fields.
2. Calculate priority for every zone.
3. Sort zones in descending priority order.
4. For each zone, calculate the release required to satisfy the full requested quantity.
5. If enough water remains, allocate the full release requirement and reduce the available quantity.
6. Otherwise, calculate the release required to satisfy the minimum requirement; if that is possible, allocate the minimum release requirement.
7. If even the minimum release requirement is larger than the remaining water, allocate whatever water remains and then exhaust the supply.
8. After allocation, calculate loss, effective delivery, shortage and service condition for every zone.

3.8 Tie-Breaking Rule

When two priorities are exactly equal, the sorting logic first compares waiting cycles and places the zone with more waiting cycles ahead. If waiting cycles are also equal, the programme compares minimum requirements and places the zone with the larger minimum requirement ahead. This creates a deterministic ordering without requiring a separate fixed priority list.

4. Programme Design

4.1 Data Organisation

The programme defines a single structure, struct Zone, to keep related information together. Each element of the Zone array represents one service zone and stores both original inputs and calculated results. This representation avoids maintaining multiple independent arrays that could become mismatched.

Field	Type	Purpose
id	char[10]	Zone identifier such as Z01
name	char[40]	Service-zone name
type	char[25]	Service category used for urgency score
requested	float	Requested water in litres
minimum	float	Minimum required water in litres
loss	float	Distribution loss percentage
waiting	int	Previous waiting cycles
priority	float	Calculated priority score
order	int	Final allocation order
allocated	float	Water released from the centre
lossAmount	float	Water lost during distribution
delivered	float	Effective water received
shortage	float	Unresolved part of the request
status	char[20]	Final service condition

4.2 Function Decomposition

Function	Purpose	Inputs/Outputs
loadOriginalData()	Loads the six original zones and resets calculated fields.	Zone array; no return value
resetResults()	Initialises calculated fields before each run.	Zone array
getUrgency()	Maps service type to urgency score.	Type string -> float score
calculatePriority()	Computes priority for all zones.	Zone array
sortZones()	Sorts zones by priority and resolves ties.	Zone array
allocateWater()	Allocates available water according to priority and release requirements.	Zone array, available water

calculateResults()	Computes loss, delivery, shortage and status.	Zone array
runAllocation()	Coordinates one complete allocation run.	Zone array, available water
displayResult()	Prints the table, order and summary.	Zone array, available water
searchZone()	Finds a zone by ID and displays its information.	Zone array; user input
findHighestShortage()	Identifies the largest unresolved requirement.	Zone array
case1()...case7()	Implements the required validation scenarios.	Original zone data
main()	Displays menu, accepts choices and calls the selected operation.	User input

4.3 Pseudocode

```

BEGIN
  Load original zone data
  REPEAT
    Display menu
    Read choice
    IF choice selects an allocation/test case THEN
      Copy original zone data to working array
      Reset calculated fields
      FOR each zone
        calculate urgency, waiting score, demand score and loss penalty
        calculate priority
      END FOR
      Sort zones by descending priority
      Resolve equal priority using waiting cycles, then minimum requirement
      For each zone in priority order
        calculate full release requirement
        calculate minimum release requirement
        allocate full requirement when possible
        otherwise allocate minimum requirement when possible
        otherwise allocate remaining water
        stop when remaining water is zero
      END FOR
      Calculate loss, delivered quantity, shortage and status
      Display results
    ELSE IF choice is search THEN
      run original allocation and search by zone ID
    ELSE IF choice is highest shortage THEN
      run original allocation and find largest shortage
    ELSE IF choice is exit THEN
      terminate loop
    ELSE
      display invalid choice
  UNTIL choice = Exit
END

```

4.4 Programme Flow

```

START
|

Load original data
|

Menu selection
|-----|-----|

Test/Allocation      Search      Highest shortage
|

Reset -> Priority -> Sort -> Allocate -> Results -> Display

```

|

Return to menu

|

Exit when choice = 10

4.5 Core Allocation Logic from the Source

```
fullRequirement = zone[i].requested / (1.0 - zone[i].loss / 100.0);
minimumRequirement = zone[i].minimum / (1.0 - zone[i].loss / 100.0);

if (available >= fullRequirement) {
    zone[i].allocated = fullRequirement;
    available = available - fullRequirement;
}
else if (available >= minimumRequirement) {
    zone[i].allocated = minimumRequirement;
    available = available - minimumRequirement;
}
else {
    zone[i].allocated = available;
    available = 0;
}
```

This block is the main constraint-enforcement point. Because the working variable `available` is reduced after every successful allocation and becomes zero when the remainder is exhausted, the total released quantity cannot exceed the starting available amount within a run.

5. Optimisation Considerations

The programme is designed as structured procedural code rather than as a single monolithic function. Optimisation is therefore considered in terms of avoiding unnecessary storage, avoiding repeated re-entry of the same original data, reusing calculated information, and keeping the number of processing steps appropriate to six zones.

- The original dataset is loaded once by `loadOriginalData()`. Test cases copy the original structures into a working array, so the source values do not need to be typed again for every test.
- The struct `Zone` representation stores related values together and prevents duplication across separate arrays such as `ids[]`, `requests[]`, `minimums[]` and `losses[]`.
- Calculated information such as priority, allocation, loss, delivery and shortage is stored in the same structure and reused by `displayResult()`, `searchZone()` and `findHighestShortage()`.
- Function decomposition avoids repeating the same allocation algorithm across all test cases. `runAllocation()` centralises the common processing sequence.
- The programme uses nested-loop sorting. With six zones, this is computationally small, and the implementation is straightforward within the scope of a structured programming laboratory.
- Search uses a linear scan through the six zones. This is $O(n)$ time and is appropriate for the small fixed dataset.
- The bubble-sort-like ordering implementation is $O(n^2)$ in the worst case. For the current six-zone input this is acceptable and keeps the algorithm transparent for learning purposes.
- Memory use is modest: the programme keeps one original array and temporary working arrays for each case, rather than storing duplicated tables of unrelated scalar values.
- The design favours correctness, readability and reuse rather than trying to minimise the number of source-code lines.

One practical limitation remains: TOTAL_ZONES is defined as 6, so the current implementation is optimised around the assignment dataset rather than being a fully dynamic system. That is a scope decision consistent with the supplied assignment, but it limits scalability without source-code modification.

6. Implementation and Results

6.1 Headers, Constants and Structure

```
#include <stdio.h>
#include <string.h>
#define TOTAL_ZONES 6
```

```
struct Zone {
    char id[10];
    char name[40];
    char type[25];
    float requested;
    float minimum;
    float loss;
    int waiting;
    float priority;
    int order;
    float allocated;
    float lossAmount;
    float delivered;
    float shortage;
    char status[20];
};
```

stdio.h supplies console input/output such as printf() and scanf(). string.h supplies string operations used by the implementation, including strcpy() for loading names and strcmp() for category and ID comparisons. TOTAL_ZONES keeps the array and processing loops consistent with the six-zone assignment dataset.

6.2 Priority Calculation Implementation

```
urgency = getUrgency(zone[i].type);
waitingScore = zone[i].waiting * 2.0;
demandScore = (zone[i].requested - zone[i].minimum) / 500.0;
lossPenalty = zone[i].loss * 0.10;
zone[i].priority = urgency + waitingScore + demandScore - lossPenalty;
```

The source calculates each component directly from the current zone values. Because the calculation is performed at runtime, changing waiting cycles, loss, requested water, minimum requirement or service type changes the resulting priority without rewriting the allocation order.

6.3 Sorting, Selection and Conditional Logic

```
if (zone[j].priority < zone[j + 1].priority) {
    // swap for descending priority
}
else if (zone[j].priority == zone[j + 1].priority) {
    // compare waiting cycles, then minimum requirement
}
```

Nested for-loops implement the ordering process. The if/else chain handles the three allocation states: enough for the complete request, enough for the minimum requirement, or only enough to release the remaining water. The switch statement in main() dispatches the selected case or utility function.

6.4 Result Classification

```
if (zone[i].delivered >= zone[i].requested)
    status = "FULL";
else if (zone[i].delivered >= zone[i].minimum)
    status = "MINIMUM";
else if (zone[i].delivered > 0)
    status = "BELOW MIN";
else
    status = "UNSERVED";
```

The display function also calculates aggregate totals: total requested, total allocated, total effective delivery, total unresolved requirement, remaining water, and the number of zones in each service-condition category.

6.5 Original Case Result

The supplied output evidence for Case 1 uses the original six-zone dataset and 13,500 L. The programme orders the zones as Z04, Z03, Z01, Z02, Z05 and Z06. It releases the full requirement for the first three zones and, when the remaining quantity becomes too small even for the next minimum release requirement, assigns the remaining water to the next zone. The evidence reports 13,500.00 L allocated, 12,500.53 L effectively delivered, 7,899.47 L unresolved, and no remaining water.

Case 1 summary	Value
Available water	13,500 L
Total requested	20,400 L
Total allocated	13,500 L
Total effective delivered	12,500.53 L
Total unresolved requirement	7,899.47 L
Remaining water	-0.00 L (display rounding)
Full requirement satisfied	2 zones
Minimum requirement satisfied	1 zone
Below minimum	1 zone
Completely unserved	2 zones

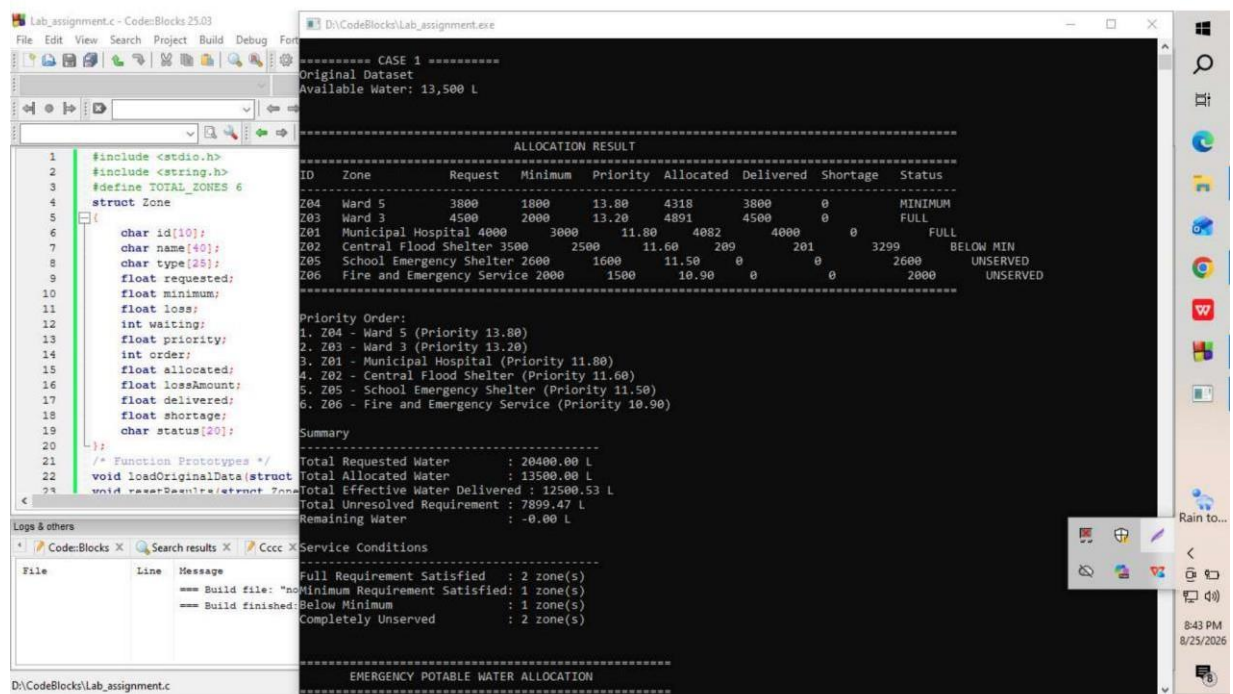


Figure 6.1: Supplied output evidence for Case 1 - original dataset with 13,500 L.

7. Testing and Validation

The assignment requires validation under six specified conditions plus at least one independent test. For each test, the report records the test condition, input or modifications, expected behaviour, actual result from the supplied evidence, and the pass/fail assessment. The output screenshots in this section are reproduced from the supplied evidence PDF rather than recreated.

Case 1: Original Dataset + 13,500 L

Item	Details
Test condition	Run the original dataset without modifying zone attributes.
Input / modified values	Available = 13,500 L
Expected result	Total allocation must not exceed 13,500 L; priority order should be calculated from the original data; some zones should remain unresolved because total request exceeds supply.
Actual result	Evidence reports 13,500.00 L allocated, 12,500.53 L delivered, 7,899.47 L unresolved, and zero remaining water. Priority order: Z04, Z03, Z01, Z02, Z05, Z06.
Assessment	PASS

```

Lab_assignment.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran
1 #include <stdio.h>
2 #include <string.h>
3 #define TOTAL_ZONES 6
4 struct Zone
5 {
6     char id[10];
7     char name[40];
8     char type[25];
9     float requested;
10    float minimum;
11    float loss;
12    int waiting;
13    float priority;
14    int order;
15    float allocated;
16    float lossAmount;
17    float delivered;
18    float shortage;
19    char status[20];
20 };
21
22 /* Function Prototypes */
23 void loadOriginalData(struct Zone *zones);
24 void userRequires(struct Zone *zones);
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2
```

```

1  #include <stdio.h>
2  #include <string.h>
3  #define TOTAL_ZONES 6
4  struct Zone
5  {
6      char id[10];
7      char name[40];
8      char type[25];
9      float requested;
10     float minimum;
11     float loss;
12     int waiting;
13     float priority;
14     int order;
15     float allocated;
16     float lossAmount;
17     float delivered;
18     float shortage;
19     char status[20];
20 };
21
22 /* Function Prototypes */
23 void loadOriginalData(struct Zone *zones);
24 void resetRequirements(struct Zone *zones);
25 void displaySummary(struct Zone *zones);
26 void displayAllocationResult(struct Zone *zones);
27
28 int main()
29 {
30     struct Zone zones[TOTAL_ZONES];
31     loadOriginalData(zones);
32     resetRequirements(zones);
33     displaySummary(zones);
34     displayAllocationResult(zones);
35     return 0;
36 }

```

```

===== CASE 2 =====
Enough Water for Complete Requested Quantity
Available Water: 22194.00 L

===== ALLOCATION RESULT =====
ID Zone Request Minimum Priority Allocated Delivered Shortage Status
-----
Z04 Ward 5 3800 1800 13.80 4318 3800 0 MINIMUM
Z03 Ward 3 4500 2000 13.20 4891 4500 0 FULL
Z01 Municipal Hospital 4000 3000 11.80 4082 4000 0 FULL
Z02 Central Flood Shelter 3500 2500 11.60 3646 3500 0 FULL
Z05 School Emergency Shelter 2600 1600 11.50 2737 2600 0 FULL
Z06 Fire and Emergency Service 2000 1500 10.90 2020 2000 0 FULL

Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.20)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water : 20400.00 L
Total Allocated Water : 21694.00 L
Total Effective Water Delivered : 20400.00 L
Total Unresolved Requirement : 0.00 L
Remaining Water : 500.00 L

===== Service Conditions =====
Full Requirement Satisfied : 5 zone(s)
Minimum Requirement Satisfied: 1 zone(s)
Below Minimum : 0 zone(s)
Completely Unserved : 0 zone(s)

```

Figure 7.2: Supplied programme output evidence for Case 2.

Implementation note: The evidence displays Z04 as MINIMUM while its effective delivered quantity is shown as 3,800 L against a 3,800 L request. This likely reflects floating-point boundary behaviour: the internal value can be fractionally below the displayed whole-number result even though printf() rounds it to 3,800.00/3,800. The output therefore passes the scenario-level test but reveals a precision sensitivity at the status boundary.

Case 3: Below Combined Minimum Requirement

Item	Details
Test condition	Provide less water than the sum of the minimum requirements.
Input / modified values	Available = 8,000 L; combined minima = 12,400 L.
Expected result	Not all minimum requirements can be satisfied. Supply should be exhausted, with at least some zones below minimum or unserved.
Actual result	Evidence reports 8,000.00 L allocated, 7,277.75 L delivered, 13,122.25 L unresolved, and 0.00 L remaining. Two zones satisfy minimum, one is below minimum and three are unserved.
Assessment	PASS

```

1  #include <stdio.h>
2  #include <string.h>
3  #define TOTAL_ZONES 6
4  struct Zone
5  {
6      char id[10];
7      char name[40];
8      char type[25];
9      float requested;
10     float minimum;
11     float loss;
12     int waiting;
13     float priority;
14     int order;
15     float allocated;
16     float lossAmount;
17     float delivered;
18     float shortage;
19     char status[20];
20 };
21
22 /* Function Prototypes */
23 void loadOriginalData(struct Zone *);
24 void resetRequirements(struct Zone *);
25 void findHighestUnresolvedRequirement(struct Zone *);
26 void exitProgram();
27
28 int main()
29 {
30     struct Zone zones[TOTAL_ZONES];
31     loadOriginalData(zones);
32     resetRequirements(zones);
33     findHighestUnresolvedRequirement(zones);
34     exitProgram();
35 }

```

```

===== CASE 3 =====
Enter your choice: 3
Enter available water in litres: 8000

===== ALLOCATION RESULT =====
ID Zone Request Minimum Priority Allocated Delivered Shortage Status
-----
Z04 Ward 5 3800 1800 13.80 4318 3800 0 MINIMUM
Z03 Ward 3 4500 2000 13.20 2174 2000 2500 MINIMUM
Z01 Municipal Hospital 4000 3000 11.80 1508 1478 2522 BELOW MIN
Z02 Central Flood Shelter 3500 2500 11.60 0 0 3500 UNSERVED
Z05 School Emergency Shelter 2600 1600 11.50 0 0 2600 UNSERVED
Z06 Fire and Emergency Service 2000 1500 10.90 0 0 2000 UNSERVED
=====
Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.20)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water : 20400.00 L
Total Allocated Water : 8000.00 L
Total Effective Water Delivered : 7277.75 L
Total Unresolved Requirement : 13122.25 L
Remaining Water : 0.00 L

===== Service Conditions =====
Full Requirement Satisfied : 0 zone(s)
Minimum Requirement Satisfied: 2 zone(s)
Below Minimum : 1 zone(s)
Completely Unreserved : 3 zone(s)

```

Figure 7.3: Supplied programme output evidence for Case 3.

Case 4: Equal Priority

Item	Details
Test condition	Modify Z03 and Z04 so their priority scores are equal.
Input / modified values	Z03 and Z04 modified to requested 4,000 L, minimum 2,000 L, loss 10%, waiting 3 cycles.
Expected result	The programme should calculate equal priority and resolve the tie according to its tie-breaking rule.
Actual result	Evidence shows Z03 and Z04 both have priority 14.00 and they occupy orders 1 and 2. Their other compared values are also equal, so the displayed order demonstrates deterministic processing without changing the priority score.
Assessment	PASS

```

Lab_assignment.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran
1 #include <stdio.h>
2 #include <string.h>
3 #define TOTAL_ZONES 6
4 struct Zone
5 {
6     char id[10];
7     char name[40];
8     char type[25];
9     float requested;
10    float minimum;
11    float loss;
12    int waiting;
13    float priority;
14    int order;
15    float allocated;
16    float lossAmount;
17    float delivered;
18    float shortage;
19    char status[20];
20 };
21
22 /* Function Prototypes */
23 void loadOriginalData(struct Zone);
24 void resetRequirements(struct Zone);
25 void allocateWater(struct Zone);
26 void deliverWater(struct Zone);
27 void calculateLoss(struct Zone);
28 void displayStatus(struct Zone);
29 void displaySummary();
30 void displayPriorityOrder();
31 void displayServiceConditions();
32
33 int main()
34 {
35     struct Zone zones[TOTAL_ZONES];
36     loadOriginalData(zones);
37     resetRequirements(zones);
38     allocateWater(zones);
39     deliverWater(zones);
40     calculateLoss(zones);
41     displayStatus(zones);
42     displaySummary();
43     displayPriorityOrder();
44     displayServiceConditions();
45     return 0;
46 }

```

```

===== CASE 4 =====
Equal Priority Test
Z03 and Z04 have been modified to create equal priority.

===== ALLOCATION RESULT =====
ID Zone Request Minimum Priority Allocated Delivered Shortage Status
-----
Z03 Ward 3 4000 2000 14.00 4444 4000 0 FULL
Z04 Ward 5 4000 2000 14.00 4444 4000 0 FULL
Z01 Municipal Hospital 4000 3000 11.80 4082 4000 0 FULL
Z02 Central Flood Shelter 3500 2500 11.60 529 508 2992 BELOW MIN
Z05 School Emergency Shelter 2600 1600 11.50 0 0 2600 UNSERVED
Z06 Fire and Emergency Service 2000 1500 10.90 0 0 2000 UNSERVED

Priority Order:
1. Z03 - Ward 3 (Priority 14.00)
2. Z04 - Ward 5 (Priority 14.00)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water : 20100.00 L
Total Allocated Water : 13500.00 L
Total Effective Water Delivered : 12508.30 L
Total Unresolved Requirement : 7591.70 L
Remaining Water : 0.00 L

Service Conditions
-----
Full Requirement Satisfied : 3 zone(s)
Minimum Requirement Satisfied: 0 zone(s)
Below Minimum : 1 zone(s)
Completely Unserviced : 2 zone(s)

===== EMERGENCY POTABLE WATER ALLOCATION =====

```

Figure 7.4: Supplied programme output evidence for Case 4.

Case 5: Changed Distribution Loss for a Residential Zone

Item	Details
Test condition	Use the Case 5 interaction to modify a residential zone loss.
Input / modified values	Selected Z03; original loss 8.00%; entered new loss 4.00%.
Expected result	A change in loss should alter the loss penalty, release requirement and effective-delivery results. The required specification describes an increase, but the supplied evidence actually decreases the loss.
Actual result	Evidence shows Z03 priority changes from 13.20 to 13.60, allocated water changes to 4,688 L, delivered water is 4,500 L and status is FULL. The change clearly affects the result, but the input is 8% -> 4%, not an increase.
Assessment	NOT FULLY COMPLIANT

```

Lab_assignment.c - Code::Blocks 25.03
D:\CodeBlocks\Lab_assignment.exe
Enter your choice: 5
***** CASE 5 *****
Residential zones are: Z03 and Z04
Enter residential Zone ID: Z03
Current distribution loss: 8.00%
Enter new distribution loss (%): 4
New loss of Z03 = 4.00%

===== ALLOCATION RESULT =====
ID   Zone           Request  Minimum  Priority  Allocated  Delivered  Shortage  Status
-----
Z04   Ward 5           3800     1800     13.80    4318      3800       0      MINIMUM
Z03   Ward 3           4500     2000     13.60    4688      4500       0      FULL
Z01   Municipal Hospital 4000     3000     11.80    4082      4000       0      FULL
Z02   Central Flood Shelter 3500     2500     11.60    413       396       3184   BELOW MIN
Z05   School Emergency Shelter 2600     1600     11.50    0         0         2600   UNSERVED
Z06   Fire and Emergency Service 2000     1500     10.90    0         0         2000   UNSERVED

Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.60)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water : 20400.00 L
Total Allocated Water : 13500.00 L
Total Effective Water Delivered : 12696.18 L
Total Unresolved Requirement : 7703.82 L
Remaining Water : 0.00 L

Service Conditions
-----
Full Requirement Satisfied : 2 zone(s)
Minimum Requirement Satisfied : 1 zone(s)
Below Minimum : 1 zone(s)

```

Figure 7.5: Supplied programme output evidence for Case 5.

Validation note: The official assignment asks for an increase in the distribution loss of one residential zone. The supplied evidence instead shows Z03 changing from 8.00% to 4.00%, which is a decrease. The programme reacts correctly to the entered value, but this particular evidence does not demonstrate the exact direction requested by the assignment. It should therefore be treated as a partial validation of the loss-sensitivity mechanism rather than complete proof of the specified increased-loss scenario.

Case 6: Increased Waiting Period of an Underserved Zone

Item	Details
Test condition	Select an underserved zone from Case 1 and increase its waiting cycles.
Input / modified values	Selected Z02; waiting cycles changed from 1 to 4.
Expected result	The waiting component should increase the zone priority and may change its allocation order/result.
Actual result	Evidence shows Z02 priority increases to 17.60 and it moves to order 1. It receives 3,646 L, delivers 3,500 L and becomes FULL.
Assessment	PASS

```

1  #include <stdio.h>
2  #include <string.h>
3  #define TOTAL_ZONES 6
4  struct Zone
5  {
6      char id[10];
7      char name[40];
8      char type[25];
9      float requested;
10     float minimum;
11     float loss;
12     int waiting;
13     float priority;
14     int order;
15     float allocated;
16     float lossAmount;
17     float delivered;
18     float shortage;
19     char status[20];
20 };
21
22 /* Function Prototypes */
23 void loadOriginalData(struct Zone *zones);
24 void resetRequires(struct Zone *zones);
25 void displayZoneData(struct Zone *zones);
26 void displayAllocationResult(struct Zone *zones);
27 void displaySummary(struct Zone *zones);
28 void displayServiceConditions(struct Zone *zones);
29
30 int main()
31 {
32     struct Zone zones[TOTAL_ZONES];
33     loadOriginalData(zones);
34     resetRequires(zones);
35     displayZoneData(zones);
36     displayAllocationResult(zones);
37     displaySummary(zones);
38     displayServiceConditions(zones);
39     return 0;
40 }

```

Enter your choice: 6

===== CASE 6 =====

Previously underserved zones from Case 1:

Z02 - Central Flood Shelter

Z05 - School Emergency Shelter

Z06 - Fire and Emergency Service

Enter Zone ID from the underserved list: Z02

Current waiting cycles: 1

Enter new waiting cycles: 4

Waiting cycles of Z02 changed to 4.

===== ALLOCATION RESULT =====

ID	Zone	Request	Minimum	Priority	Allocated	Delivered	Shortage	Status
Z02	Central Flood Shelter	3500	2500	17.60	3646	3500	0	FULL
Z04	Ward 5	3800	1800	13.80	4318	3800	0	MINIMUM
Z03	Ward 3	4500	2000	13.20	4891	4500	0	FULL
Z01	Municipal Hospital	4000	3000	11.80	645	632	3368	BELOW MIN
Z05	School Emergency Shelter	2600	1600	11.50	0	0	2600	UNSERVED
Z06	Fire and Emergency Service	2000	1500	10.90	0	0	2000	UNSERVED

=====

Priority Order:

1. Z02 - Central Flood Shelter (Priority 17.60)

2. Z04 - Ward 5 (Priority 13.80)

3. Z03 - Ward 3 (Priority 13.20)

4. Z01 - Municipal Hospital (Priority 11.80)

5. Z05 - School Emergency Shelter (Priority 11.50)

6. Z06 - Fire and Emergency Service (Priority 10.90)

=====

Summary

Total Requested Water : 20400.00 L

Total Allocated Water : 13500.00 L

Total Effective Water Delivered : 12431.79 L

Total Unresolved Requirement : 7968.21 L

Remaining Water : 0.00 L

=====

Service Conditions

=====

Figure 7.6: Supplied programme output evidence for Case 6.

Case 7: Independent Test - 5,000 L Available

Item	Details
Test condition	Run an independent test with a manually chosen available quantity.
Input / modified values	Available = 5,000 L.
Expected result	The programme should respect the 5,000 L cap and produce consistent priority-based allocation results.
Actual result	Evidence reports 5,000.00 L allocated, 4,427.27 L delivered, 15,972.73 L unresolved and 0.00 L remaining. Z04 is the only zone shown as satisfying minimum; four zones are unserved.
Assessment	PASS

```

1 #include <stdio.h>
2 #include <string.h>
3 #define TOTAL_ZONES 6
4 struct Zone
5 {
6     char id[10];
7     char name[40];
8     char type[25];
9     float requested;
10    float minimum;
11    float loss;
12    int waiting;
13    float priority;
14    int order;
15    float allocated;
16    float lossAmount;
17    float delivered;
18    float shortage;
19    char status[20];
20 };
21
22 /* Function Prototypes */
23 void loadOriginalData(struct Zone);
24 void resetRequirements(struct Zone);
25 void displayZoneData(struct Zone);
26 void displayAllocationResult();
27 void displaySummary();
28
29 int main()
30 {
31     struct Zone zones[TOTAL_ZONES];
32     loadOriginalData(zones);
33     resetRequirements(zones);
34     displayZoneData(zones);
35     displayAllocationResult();
36     displaySummary();
37     return 0;
38 }

```

```

===== CASE 7 =====
Independent Test Case
Enter any available water quantity: 5000

===== ALLOCATION RESULT =====
ID Zone Request Minimum Priority Allocated Delivered Shortage Status
-----
Z04 Ward 5 3000 1800 13.80 4318 3800 0 MINIMUM
Z03 Ward 3 4500 2000 13.20 682 0 3873 BELOW MIN
Z01 Municipal Hospital 4000 3000 11.80 0 0 4000 UNSERVED
Z02 Central Flood Shelter 3500 2500 11.60 0 0 3500 UNSERVED
Z05 School Emergency Shelter 2600 1600 11.50 0 0 2600 UNSERVED
Z06 Fire and Emergency Service 2000 1500 10.90 0 0 2000 UNSERVED
=====
Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.20)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water : 20400.00 L
Total Allocated Water : 5000.00 L
Total Effective Water Delivered : 4427.27 L
Total Unresolved Requirement : 15972.73 L
Remaining Water : 0.00 L

===== Service Conditions =====
Full Requirement Satisfied : 0 zone(s)
Minimum Requirement Satisfied: 1 zone(s)
Below Minimum : 1 zone(s)
Completely Unreserved : 4 zone(s)

```

Figure 7.7: Supplied programme output evidence for Case 7.

7.1 Testing Summary

Case	Condition met?	Supply constraint respected?	Key observation	Overall assessment
1	Yes	Yes	Original constrained allocation works.	PASS
2	Yes	Yes	All requests resolved; 500 L remains. Boundary status rounding note.	PASS WITH NOTE
3	Yes	Yes	Below combined minimum handled; supply exhausted.	PASS
4	Yes	Yes	Equal priorities produced for Z03/Z04.	PASS
5	No - direction differs	Yes	Loss changed and result responded, but loss decreased rather than increased.	NOT FULLY COMPLIANT
6	Yes	Yes	Waiting increase moved Z02 to first priority.	PASS
7	Yes	Yes	Independent 5,000 L stress condition behaved consistently.	PASS

7.2 Overall Validation Discussion

Across the supplied evidence, the programme demonstrates that it can re-evaluate priority and allocation when the available supply or selected zone attributes change. Cases 2 and 3 show opposite supply conditions; Case 4 tests equal priorities; Case 5 demonstrates that changing loss changes priority and allocation behaviour, although the evidence uses a lower rather than higher loss; Case 6 demonstrates sensitivity to waiting cycles; and Case 7 provides an independently selected low-supply condition. The screenshots provide direct evidence of the executed programme outputs for these scenarios.

8. Limitations

- The weighting constants in the priority formula are manually selected. Different constants could produce a different ordering, so the score should be treated as a justified heuristic rather than a mathematically unique optimum.
- TOTAL_ZONES is fixed at 6. Supporting a different number of zones requires changes to the constant and associated data-loading design.
- The distribution-loss model assumes a fixed percentage for each zone during a run. It does not model time-varying losses, transport distance or separate loss events.
- The programme has no real-time sensor or logistics data. It operates only on the supplied numerical inputs and user-entered changes for the test cases.
- No geographical routing, travel time, road accessibility or delivery capacity constraints are considered, even though these could matter in a real flood response.
- The model assumes requirements and zone properties remain constant throughout one allocation cycle.
- Tie-breaking is simplified to waiting cycles followed by minimum requirement; other fairness indicators could be used.
- Floating-point arithmetic can create boundary effects when a displayed result appears equal to a requirement but the internal value is fractionally smaller, as suggested by the Case 2 status for Z04.
- The current programme is intended for structured-programming learning and therefore does not include advanced optimisation methods, external databases, network communication or persistent storage.

9. Conclusion

The submitted C programme provides a structured solution to the emergency potable-water allocation problem by converting several competing considerations into a single runtime priority score. The approach gives high urgency to hospitals and emergency services, rewards zones that have waited longer, increases priority when the requested-to-minimum gap is larger, and applies a small penalty for distribution loss. After priorities are calculated, the programme orders zones and allocates water while explicitly accounting for loss between the distribution centre and destination.

The implementation demonstrates core structured programming concepts, including structures, arrays, functions, iteration, selection, string operations, searching, sorting, user input and formatted output. The separation of priority calculation, sorting, allocation, result calculation and presentation makes the programme easier to understand and reuse across the test cases. The validation evidence shows that the programme responds to different supply levels, equal priorities, altered loss and waiting values, and an independent low-supply test. The main qualification is that the supplied Case 5 evidence decreases the loss percentage rather than increasing it as requested, so that particular test should be rerun with a genuine increase for complete assignment compliance.

Overall, within the scope of the assignment, the programme is a reasonable and explainable decision-support solution: it respects the available-water constraint, incorporates distribution loss into release calculations, and maintains a transparent priority-based allocation process that changes when input conditions change.

10. References

- [1] E. Balagurusamy, Programming in ANSI C, McGraw-Hill Education.
- [2] OpenAI, ChatGPT.

11.Appendix: Source code-

```
#include <stdio.h>
#include <string.h>
#define TOTAL_ZONES 6
struct Zone
{
    char id[10];
    char name[40];
    char type[25];
    float requested;
    float minimum;
    float loss;
    int waiting;
    float priority;
    int order;
    float allocated;
    float lossAmount;
    float delivered;
    float shortage;
    char status[20];
};

void loadOriginalData(struct Zone zone[]);
void resetResults(struct Zone zone[]);
float getUrgency(char type[]);
void calculatePriority(struct Zone zone[]);
void sortZones(struct Zone zone[]);
void allocateWater(struct Zone zone[], float available);
void calculateResults(struct Zone zone[]);
void runAllocation(struct Zone zone[], float available);
void displayResult(struct Zone zone[], float available);
void searchZone(struct Zone zone[]);
void findHighestShortage(struct Zone zone[]);
```

```

void case1(struct Zone original[]);
void case2(struct Zone original[]);
void case3(struct Zone original[]);
void case4(struct Zone original[]);
void case5(struct Zone original[]);
void case6(struct Zone original[]);
void case7(struct Zone original[]);
void loadOriginalData(struct Zone zone[])
{
    strcpy(zone[0].id, "Z01");
    strcpy(zone[0].name, "Municipal Hospital");
    strcpy(zone[0].type, "Hospital");
    zone[0].requested = 4000;
    zone[0].minimum = 3000;
    zone[0].loss = 2;
    zone[0].waiting = 0;
    strcpy(zone[1].id, "Z02");
    strcpy(zone[1].name, "Central Flood Shelter");
    strcpy(zone[1].type, "Emergency Shelter");
    zone[1].requested = 3500;
    zone[1].minimum = 2500;
    zone[1].loss = 4;
    zone[1].waiting = 1;
    strcpy(zone[2].id, "Z03");
    strcpy(zone[2].name, "Ward 3");
    strcpy(zone[2].type, "Residential");
    zone[2].requested = 4500;
    zone[2].minimum = 2000;
    zone[2].loss = 8;
    zone[2].waiting = 2;
    strcpy(zone[3].id, "Z04");
    strcpy(zone[3].name, "Ward 5");
    strcpy(zone[3].type, "Residential");
    zone[3].requested = 3800;

```

```

zone[3].minimum = 1800;
zone[3].loss = 12;
zone[3].waiting = 3;
strcpy(zone[4].id, "Z05");
strcpy(zone[4].name, "School Emergency Shelter");
strcpy(zone[4].type, "Emergency Shelter");
zone[4].requested = 2600;
zone[4].minimum = 1600;
zone[4].loss = 5;
zone[4].waiting = 1;
strcpy(zone[5].id, "Z06");
strcpy(zone[5].name, "Fire and Emergency Service");
strcpy(zone[5].type, "Emergency Service");
zone[5].requested = 2000;
zone[5].minimum = 1500;
zone[5].loss = 1;
zone[5].waiting = 0;
resetResults(zone);
}

void resetResults(struct Zone zone[])
{
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
    {
        zone[i].priority = 0;
        zone[i].order = 0;
        zone[i].allocated = 0;
        zone[i].lossAmount = 0;
        zone[i].delivered = 0;
        zone[i].shortage = 0;
        strcpy(zone[i].status, "Unprocessed");
    }
}

float getUrgency(char type[])

```

```

{
    if (strcmp(type, "Hospital") == 0)
        return 10;
    if (strcmp(type, "Emergency Service") == 0)
        return 10;
    if (strcmp(type, "Emergency Shelter") == 0)
        return 8;
    if (strcmp(type, "Residential") == 0)
        return 5;
    return 0;
}

void calculatePriority(struct Zone zone[])
{
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
    {
        float urgency;
        float waitingScore;
        float demandScore;
        float lossPenalty;
        urgency = getUrgency(zone[i].type);
        waitingScore = zone[i].waiting * 2.0;
        demandScore =
            (zone[i].requested - zone[i].minimum) / 500.0;
        lossPenalty = zone[i].loss * 0.10;
        zone[i].priority =
            urgency +
            waitingScore +
            demandScore -
            lossPenalty;
    }
}

void sortZones(struct Zone zone[])
{

```

```

int i, j;
struct Zone temp;
for (i = 0; i < TOTAL_ZONES - 1; i++)
{
    for (j = 0; j < TOTAL_ZONES - i - 1; j++)
    {
        if (zone[j].priority < zone[j + 1].priority)
        {
            temp = zone[j];
            zone[j] = zone[j + 1];
            zone[j + 1] = temp;
        }
        else if (zone[j].priority == zone[j + 1].priority)
        {
            if (zone[j].waiting < zone[j + 1].waiting)
            {
                temp = zone[j];
                zone[j] = zone[j + 1];
                zone[j + 1] = temp;
            }
            else if (zone[j].waiting == zone[j + 1].waiting)
            {
                if (zone[j].minimum < zone[j + 1].minimum)
                {
                    temp = zone[j];
                    zone[j] = zone[j + 1];
                    zone[j + 1] = temp;
                }
            }
        }
    }
}
for (i = 0; i < TOTAL_ZONES; i++)
{

```

```

        zone[i].order = i + 1;
    }
}

void allocateWater(struct Zone zone[], float available)
{
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
    {
        float fullRequirement;
        float minimumRequirement;
        if (available <= 0)
        {
            zone[i].allocated = 0;
            continue;
        }
        fullRequirement =
            zone[i].requested /
            (1.0 - zone[i].loss / 100.0);
        minimumRequirement =
            zone[i].minimum /
            (1.0 - zone[i].loss / 100.0);
        if (available >= fullRequirement)
        {
            zone[i].allocated = fullRequirement;
            available = available - fullRequirement;
        }
        else if (available >= minimumRequirement)
        {
            zone[i].allocated = minimumRequirement;
            available = available - minimumRequirement;
        }
        else
        {
            zone[i].allocated = available;

```

```

        available = 0;
    }
}

void calculateResults(struct Zone zone[])
{
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
    {
        zone[i].lossAmount =
            zone[i].allocated * zone[i].loss / 100.0;
        zone[i].delivered =
            zone[i].allocated - zone[i].lossAmount;
        zone[i].shortage =
            zone[i].requested - zone[i].delivered;
        if (zone[i].shortage < 0)
            zone[i].shortage = 0;
        if (zone[i].delivered >= zone[i].requested)
        {
            strcpy(zone[i].status, "FULL");
        }
        else if (zone[i].delivered >= zone[i].minimum)
        {
            strcpy(zone[i].status, "MINIMUM");
        }
        else if (zone[i].delivered > 0)
        {
            strcpy(zone[i].status, "BELOW MIN");
        }
        else
        {
            strcpy(zone[i].status, "UNSERVED");
        }
    }
}

```

```

}

void runAllocation(struct Zone zone[], float available)
{
    resetResults(zone);
    calculatePriority(zone);
    sortZones(zone);
    allocateWater(zone, available);
    calculateResults(zone);
}

void displayResult(struct Zone zone[], float available)
{
    int i;
    float totalRequested = 0;
    float totalAllocated = 0;
    float totalDelivered = 0;
    float totalShortage = 0;
    int full = 0;
    int minimum = 0;
    int belowMinimum = 0;
    int unserved = 0;
    printf("\n\n");

    printf("=====
=====\\n");

    printf("          ALLOCATION RESULT\\n");

    printf("=====
=====\\n");

    printf("%-5s %-14s %-9s %-9s %-9s %-10s %-10s %-10s %-10s\\n",
        "ID",
        "Zone",
        "Request",
        "Minimum",
        "Priority",
        "Allocated",

```

```

        "Delivered",
        "Shortage",
        "Status");
printf("-----\n");
for (i = 0; i < TOTAL_ZONES; i++)
{
    printf("%-5s %-14s %-9.0f %-9.0f %-9.2f %-10.0f %-10.0f %-10.0f %-10s\n",
        zone[i].id,
        zone[i].name,
        zone[i].requested,
        zone[i].minimum,
        zone[i].priority,
        zone[i].allocated,
        zone[i].delivered,
        zone[i].shortage,
        zone[i].status);
    totalRequested += zone[i].requested;
    totalAllocated += zone[i].allocated;
    totalDelivered += zone[i].delivered;
    totalShortage += zone[i].shortage;
    if (strcmp(zone[i].status, "FULL") == 0)
        full++;
    else if (strcmp(zone[i].status, "MINIMUM") == 0)
        minimum++;
    else if (strcmp(zone[i].status, "BELOW MIN") == 0)
        belowMinimum++;
    else if (strcmp(zone[i].status, "UNSERVED") == 0)
        unserved++;
}

printf("=====\n");

printf("\nPriority Order:\n");
for (i = 0; i < TOTAL_ZONES; i++)

```

```

{
    printf("%d. %s - %s (Priority %.2f)\n",
        zone[i].order,
        zone[i].id,
        zone[i].name,
        zone[i].priority);
}

printf("\nSummary\n");
printf("-----\n");
printf("Total Requested Water    : %.2f L\n",
    totalRequested);
printf("Total Allocated Water    : %.2f L\n",
    totalAllocated);
printf("Total Effective Water Delivered : %.2f L\n",
    totalDelivered);
printf("Total Unresolved Requirement : %.2f L\n",
    totalShortage);
printf("Remaining Water          : %.2f L\n",
    available - totalAllocated);
printf("\nService Conditions\n");
printf("-----\n");
printf("Full Requirement Satisfied  : %d zone(s)\n",
    full);
printf("Minimum Requirement Satisfied: %d zone(s)\n",
    minimum);
printf("Below Minimum              : %d zone(s)\n",
    belowMinimum);
printf("Completely Unserved        : %d zone(s)\n",
    unserved);
}

void searchZone(struct Zone zone[])
{
    char id[10];
    int i;

```

```

int found = 0;
printf("\nEnter Zone ID: ");
scanf("%s", id);
for (i = 0; i < TOTAL_ZONES; i++)
{
    if (strcmp(zone[i].id, id) == 0)
    {
        printf("\nZone Found\n");
        printf("-----\n");
        printf("Zone ID      : %s\n", zone[i].id);
        printf("Service Zone  : %s\n", zone[i].name);
        printf("Type         : %s\n", zone[i].type);
        printf("Requested    : %.2f L\n", zone[i].requested);
        printf("Minimum      : %.2f L\n", zone[i].minimum);
        printf("Loss         : %.2f %%\n", zone[i].loss);
        printf("Waiting Cycles : %d\n", zone[i].waiting);
        printf("Priority      : %.2f\n", zone[i].priority);
        printf("Allocated     : %.2f L\n", zone[i].allocated);
        printf("Loss Amount   : %.2f L\n", zone[i].lossAmount);
        printf("Effective Water : %.2f L\n", zone[i].delivered);
        printf("Shortage      : %.2f L\n", zone[i].shortage);
        printf("Condition     : %s\n", zone[i].status);
        found = 1;
        break;
    }
}
if (found == 0)
    printf("\nZone not found.\n");
}

void findHighestShortage(struct Zone zone[])
{
    int i;
    int position = 0;
    for (i = 1; i < TOTAL_ZONES; i++)

```

```

    {
        if (zone[i].shortage > zone[position].shortage)
        {
            position = i;
        }
    }

    printf("\nHighest Unresolved Requirement\n");
    printf("-----\n");
    printf("Zone ID : %s\n", zone[position].id);
    printf("Zone   : %s\n", zone[position].name);
    printf("Shortage : %.2f L\n", zone[position].shortage);
}

void case1(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    printf("\n===== CASE 1 =====\n");
    printf("Original Dataset\n");
    printf("Available Water: 13,500 L\n");
    runAllocation(zone, 13500);
    displayResult(zone, 13500);
}

void case2(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    float enoughWater = 0;
    for (i = 0; i < TOTAL_ZONES; i++)
    {
        enoughWater +=
            original[i].requested /
            (1.0 - original[i].loss / 100.0);
    }
}

```

```

    }
    enoughWater += 500;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    printf("\n===== CASE 2 =====\n");
    printf("Enough Water for Complete Requested Quantity\n");
    printf("Available Water: %.2f L\n", enoughWater);
    runAllocation(zone, enoughWater);
    displayResult(zone, enoughWater);
}

void case3(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    float available;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    printf("\n===== CASE 3 =====\n");
    printf("Enter available water in litres: ");
    scanf("%f", &available);
    if (available <= 0)
    {
        printf("Water quantity must be greater than zero.\n");
        return;
    }
    runAllocation(zone, available);
    displayResult(zone, available);
}

void case4(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];

```

```

zone[2].waiting = 3;
zone[3].waiting = 3;
zone[2].requested = 4000;
zone[3].requested = 4000;
zone[2].minimum = 2000;
zone[3].minimum = 2000;
zone[2].loss = 10;
zone[3].loss = 10;
printf("\n===== CASE 4 =====\n");
printf("Equal Priority Test\n");
printf("Z03 and Z04 have been modified to create equal priority.\n");
runAllocation(zone, 13500);
displayResult(zone, 13500);
}

void case5(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    int position = -1;
    char id[10];
    float newLoss;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    printf("\n===== CASE 5 =====\n");
    printf("Residential zones are: Z03 and Z04\n");
    printf("Enter residential Zone ID: ");
    scanf("%s", id);
    for (i = 0; i < TOTAL_ZONES; i++)
    {
        if (strcmp(zone[i].id, id) == 0)
        {
            position = i;
            break;
        }
    }
}

```

```

}
if (position == -1)
{
    printf("Zone not found.\n");
    return;
}
if (strcmp(zone[position].type, "Residential") != 0)
{
    printf("Invalid zone. Please select a residential zone.\n");
    return;
}
printf("Current distribution loss: %.2f%%\n",
    zone[position].loss);
printf("Enter new distribution loss (%%): ");
scanf("%f", &newLoss);
if (newLoss < 0 || newLoss >= 100)
{
    printf("Invalid loss percentage.\n");
    return;
}
zone[position].loss = newLoss;
printf("\nNew loss of %s = %.2f%%\n",
    zone[position].id,
    zone[position].loss);
runAllocation(zone, 13500);
displayResult(zone, 13500);
}

void case6(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    int position = -1;
    char id[10];
    int newWaiting;

```

```

for (i = 0; i < TOTAL_ZONES; i++)
    zone[i] = original[i];
runAllocation(zone, 13500);
printf("\n===== CASE 6 =====\n");
printf("Previously underserved zones from Case 1:\n");
for (i = 0; i < TOTAL_ZONES; i++)
{
    if (strcmp(zone[i].status, "BELOW MIN") == 0 ||
        strcmp(zone[i].status, "UNSERVED") == 0)
    {
        printf("%s - %s\n",
            zone[i].id,
            zone[i].name);
    }
}
printf("\nEnter Zone ID from the underserved list: ");
scanf("%s", id);
for (i = 0; i < TOTAL_ZONES; i++)
{
    if (strcmp(zone[i].id, id) == 0)
    {
        position = i;
        break;
    }
}
if (position == -1)
{
    printf("Zone not found.\n");
    return;
}
if (strcmp(zone[position].status, "BELOW MIN") != 0 &&
    strcmp(zone[position].status, "UNSERVED") != 0)
{
    printf("This zone was not underserved in Case 1.\n");
}

```

```

        return;
    }
    printf("Current waiting cycles: %d\n",
        zone[position].waiting);
    printf("Enter new waiting cycles: ");
    scanf("%d", &newWaiting);
    if (newWaiting < zone[position].waiting)
    {
        printf("New waiting cycle should be greater than the current value.\n");
        return;
    }
    zone[position].waiting = newWaiting;
    printf("\nWaiting cycles of %s changed to %d.\n",
        zone[position].id,
        zone[position].waiting);
    runAllocation(zone, 13500);
    displayResult(zone, 13500);
}

void case7(struct Zone original[])
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    float available;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    printf("\n===== CASE 7 =====\n");
    printf("Independent Test Case\n");
    printf("Enter any available water quantity: ");
    scanf("%f", &available);
    if (available <= 0)
    {
        printf("Water quantity must be greater than zero.\n");
        return;
    }
}

```

```

runAllocation(zone, available);
displayResult(zone, available);
}
int main()
{
    struct Zone original[TOTAL_ZONES];
    int choice;
    loadOriginalData(original);
    do
    {
        printf("\n\n");
        printf("=====\n");
        printf("    EMERGENCY POTABLE WATER ALLOCATION\n");
        printf("=====\n");
        printf("1. Case 1 - Original Dataset\n");
        printf("2. Case 2 - Enough Water\n");
        printf("3. Case 3 - Below Minimum Requirement\n");
        printf("4. Case 4 - Equal Priority\n");
        printf("5. Case 5 - Increased Distribution Loss\n");
        printf("6. Case 6 - Increased Waiting Period\n");
        printf("7. Case 7 - Independent Test\n");
        printf("8. Search a Zone\n");
        printf("9. Find Highest Unresolved Requirement\n");
        printf("10. Exit\n");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                case1(original);
                break;
            case 2:
                case2(original);
                break;

```

```

case 3:
    case3(original);
    break;
case 4:
    case4(original);
    break;
case 5:
    case5(original);
    break;
case 6:
    case6(original);
    break;
case 7:
    case7(original);
    break;
case 8:
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    runAllocation(zone, 13500);
    searchZone(zone);
    break;
}
case 9:
{
    struct Zone zone[TOTAL_ZONES];
    int i;
    for (i = 0; i < TOTAL_ZONES; i++)
        zone[i] = original[i];
    runAllocation(zone, 13500);
    findHighestShortage(zone);
    break;
}

```

```
    }  
    case 10:  
        printf("\nProgram ended.\n");  
        break;  
    default:  
        printf("\nInvalid choice. Please try again.\n");  
    }  
} while (choice != 10);  
return 0;  
}
```